AWS TERRAFORM INFRASTRUCTURE PROJECT

A Practical Guide — Modular Infrastructure as Code on AWS

Akash Kolhe | Cloud & DevOps Engineer
GitHub • LinkedIn • Project Repo

1. Objective

This practical demonstrates how to provision AWS infrastructure using a modular Terraform architecture instead of one large configuration file. Every resource — VPC, Security Group, EC2, and S3 — is built as an independent, reusable module and wired together through an environment-specific root configuration.

Resources Provisioned

- Amazon VPC with public subnet
- Internet Gateway + Route Table
- Security Group (SSH, HTTP, HTTPS)
- EC2 instance (Ubuntu)
- S3 bucket

2. Architecture

The root module in environments/dev calls each child module in sequence: VPC first, then Security Group, then EC2 (attached to that security group), with S3 provisioned independently.

```
Developer
   |
   |  terraform apply
   v
Terraform Root Module (environments/dev)
   |
   |-- VPC Module
   |-- Security Group Module --> EC2 Module
   |-- S3 Module
```

3. Repository Structure

```
terraform-infra-project/
├── environments/dev/
│   ├── backend.tf
│   ├── main.tf
```

```
│   ├── provider.tf
│   ├── terraform.tfvars
│   ├── variables.tf
│   └── output.tf
├── modules/
│   ├── ec2/
│   ├── s3/
│   ├── security_group/
│   └── vpc/
├── .gitignore
└── README.md
```

4. Environment Setup

4.1 Install AWS CLI

```
sudo apt install unzip
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
aws configure
```

aws configure prompts for your Access Key ID, Secret Access Key, default region, and output format — required before Terraform can authenticate to AWS.

4.2 Install Terraform

```
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-
release || lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update
sudo apt install terraform
terraform --version
```

4.3 Install Git

```
sudo apt install git
git --version
```

5. Terraform Modules

Module	Creates
VPC	Custom VPC, public subnet, Internet Gateway,

	route table, route table association
Security Group	SSH (22), HTTP (80), HTTPS (443) inbound rules, all outbound traffic
EC2	Ubuntu instance, key pair association, security group attachment
S3	S3 bucket with bucket configuration

6. Deployment Workflow

Navigate to the environment directory — this is the root module that wires all four child modules together:

```
cd ~/terraform-infra-project/environments/dev
```

Run the standard Terraform lifecycle:

```
terraform init        # downloads providers, sets up backend
terraform validate    # checks syntax and internal consistency
terraform plan        # previews the exact resources to be created
terraform apply       # provisions the infrastructure on AWS
```

Verify the created resources directly in the AWS Console: VPC, EC2 instance, Security Group, and S3 bucket.

Tear Down

```
terraform destroy
```

Always destroy learning/demo infrastructure once you're done verifying it, to avoid ongoing AWS charges.

7. Variables & Outputs

Variables are defined in terraform.tfvars, keeping the code environment-agnostic:

```
aws_region     = "us-east-1"
instance_type  = "t3.micro"
bucket_name    = "terraform-demo-bucket"
vpc_cidr       = "10.0.0.0/16"
```

Terraform outputs after apply:

- Instance ID
- Public IP
- VPC ID

- Security Group ID
- Bucket Name

8. Inspecting the Project

View the full folder tree (excluding noise like .terraform state caches):

```
cd ~/terraform-infra-project
tree -I '.terraform|.git|node_modules|venv|__pycache__'
```

Check the on-disk size of the project:

```
cd terraform-infra-project
du -hsc *
```

9. Version Control with Git

```
git init
git config --global user.name "Akash Kolhe"
git config --global user.email "akashkolhe021@gmail.com"
git branch -M main

git remote add origin https://github.com/akashkolhe/aws-terraform-infra-
project.git

git add modules environments
git commit -m "Terraform-aws-infrastructure module wise project"
git push -u origin main
```

10. Packaging for Sharing / Backup

```
sudo apt update
sudo apt install zip -y
cd /home/ubuntu
zip -r terraform-infra-project.zip terraform-infra-project
```

11. Best Practices Followed

- Modular design with reusable modules
- Environment separation (environments/dev)
- Variables instead of hardcoded values
- Outputs for cross-resource/reference sharing
- Version-controlled infrastructure via Git

- Clean, predictable folder structure
- Remote-backend ready (backend.tf)

12. Key Learnings

- Designing Terraform modules for reuse across environments
- Managing infrastructure state through variables and outputs
- The full lifecycle: init → validate → plan → apply → destroy
- Structuring a DevOps repo the way real teams do it
- Packaging and version-controlling infrastructure code professionally

13. Future Improvements

- Auto Scaling Group + Load Balancer
- NAT Gateway with private subnets
- Multi-AZ deployment
- RDS integration
- CloudWatch monitoring & alarms
- IAM roles (least privilege)
- Route 53 + ACM for DNS/TLS
- CI/CD pipeline via GitHub Actions
- Terraform Workspaces for multi-environment management

14. Technologies Used

Terraform • AWS EC2 • AWS VPC • AWS S3 • Security Groups • Ubuntu • Git • GitHub

---

Prepared by Akash Kolhe — Cloud & DevOps Engineer